Problem Understanding Paper:
A Reproducible Distributed Systems Lab Platform

Pragyam Tiwari

October 18, 2026

1 What problem are you investigating?

A distributed system is a program split into several parts that run on different machines and talk over a network. In a distributed systems course, students run a small version on their own laptops and watch what happens when something fails. The problem is that two students can follow the same instructions and get different results. Then a student can't tell whether the difference comes from their code or their computer.

My example is a small database stored on three machines: a main copy (the primary) and two backups (replicas). Each machine runs as a container, a lightweight package holding a program and what it needs to run.

Normal behavior: a student saves a value to the primary, the primary sends it to both replicas, and any of them returns the new value. **Failure part:** the student cuts the network link between the primary and one replica for ten seconds while values keep being saved. The lesson is that the cut-off replica gives old answers, then catches up.

What could go wrong:

- A replica starts before the primary is ready, so it fails to sync even before the triggered fault.
- Students cut the link at different moments causing timing errors.
- Old data from an earlier run is still there, so the lab doesn't start clean and causes confusion.

Boundaries: one exercise (this database), one failure (a network cut), and students running the lab on their own computers with Docker, a common tool for running containers.

Why this is bigger than one lab. This lab is one small case of a wider difficulty in learning distributed systems. The interesting behavior in these systems, such as delays, lost messages, and machines going down, depends on timing and on the environment, so it can change from run to run. This is my own reasoning. It means students often can't tell a mistake in their thinking from a quirk of their setup, and the same issue would show up in other exercises, not just mine. The Labtainers paper makes a related point about lab exercises in general: different operating environments and infrastructure needs make labs hard to share and keep working [2]. The National Science Foundation's CyberTraining program also asks for training materials that address unresolved bottlenecks in preparing the research workforce [1].

2 Who or what is affected by it?

Students may spend hours debugging code that is fine, or learn the wrong lesson. A student whose replica never started might think replicas don't catch up after a cut. Mistakes in lab procedure are often not caught as the program believes them to be intentional expected behvaior which can be confusing for students.

Instructors can't describe one expected result for the class, so they can't fairly compare work, and they end up fixing machines one at a time. The exercise itself only teaches its lesson if events happen in the same order every time.

3 What evidence do you currently have that it is a real problem?

Published opportunity. The National Science Foundation (NSF) runs a program called Training-based Workforce Development for Advanced Cyberinfrastructure (CyberTraining). Cyberinfrastructure means advanced computing, large-scale data handling, and networking and security for large systems. The program funds training and teaching materials, with pilot projects up to $300,000 and larger ones up to $1,000,000, and its next full-proposal deadline is January 21, 2027. Source: https://www.nsf.gov/funding/opportunities/training-based-workforce-development-advanced

This shows there is funding for training tools like this. It doesn't mention containers or failure experiments, so it doesn't prove my specific problem but shows the demand for easier training of distributed computing in general.

Technical sources. A 2017 paper on Labtainers, a Docker-based framework for cybersecurity labs, names infrastructure requirements and different operating environments as obstacles to sharing lab exercises. It recommends Docker for such exercises, claiming the software gives a consistent environment. Source: https://usenix.org/conference/ase17/workshop-program/presentation/irvine

Docker's documentation says that on startup, Compose (its tool for running several containers together) waits only until a container is running, not until it is ready, unless a health check is added. Source: https://docs.docker.com/compose/how-tos/startup-order/

My own observation. I ran a small Docker Compose setup with two services, where the second depended on the first. I saw the second service start while the first was not yet ready. This matches the documentation and my first guess above.

Assumptions. Students use their own laptops, instructors want one expected result per exercise, and the timing of the failure changes what students see.

4 What solutions already exist?

1. Setting up by hand. Each student starts every part and cuts the connection with a typed command. This needs no extra tools, but every step can differ and the failure's timing depends on when the student types. These manual interactions and setup is prone to error.

2. A shared Docker Compose file. Compose describes an app made of several containers in one YAML file (a plain text settings format), listing services (programs), networks, and volumes (storage). One command starts everything. Source: https://docs.docker.com/compose/intro/compose-application-model/

Strength: every student gets the same services set up the same way. Limits: as I observed, it doesn't wait for readiness unless a health check is written unless calibrated to do so. Also, `docker compose down` removes containers and networks by default but keeps named volumes unless `-v` is added, so old data can survive into the next run. Source: https://docs.docker.com/reference/cli/docker/compose/down/ The Compose pages I read don't describe cutting a connection at a set time or recording results which is essential for learning.

3. Compose plus a failure tool.

Toxiproxy is a Transmission Control Protocol (TCP) proxy, a middleman between two programs, controlled over HTTP (HyperText Transfer Protocol). A proxy can be switched off to cut a connection, and it can add delay or timeouts. Limit: the application must be pointed at the proxy instead of the real service, which changes the lab's setup. Source: https://github.com/Shopify/toxiproxy

Pumba is a chaos testing tool for Docker containers. It can stop or pause containers and add network delay and packet loss. Its documentation says it targets Linux and doesn't support Windows which makes it inaccessible for many students. Source: https://github.com/alexei-led/pumba

What's missing. Hand setup gives control but no consistency. Compose gives consistent setup but no failure timing or recording. The failure tools give timing but not readiness checks or recorded results. I didn't find one option that starts the system, waits until it's ready, causes the failure on time, records what happened, and cleans up.

5 What is the most important thing you still do not know?

Which setup details must be controlled so the failure gives comparable results? Candidates are readiness, when the cut starts and how long it lasts, software versions, resources given to each container, type of computer, and leftover data. Control too little and results still differ. Control too much and the tool gets complicated.

A research step: run the same lab many times, changing one detail at a time. Measure how many values were saved during the cut, how many the cut-off replica was missing at the end, how long it took to catch up, and whether all three machines finally agreed. A detail matters if changing it moves these numbers more than normal run-to-run variation. Details that matter would be fixed or recorded by my design.

6 Your Individual Scope

I will study one exercise (the three-machine database) and one failure (a ten-second network cut between the primary and one replica). I will exclude other exercises, other failures, cloud setups, grading, and a user interface.

My comparison is between the three approaches above, judged on setup consistency, control of failure timing, result recording, and cleanup. I used documentation, published papers, and one small Compose test. For the design stage I expect to use Docker, Compose, and one failure tool such as Toxiproxy on a normal laptop.

A possible direction, not a proven answer, is a "runner" program that starts services from a fixed Compose file, waits for health checks, causes the failure at a set time, saves results to a file, and cleans up.

This is manageable for a solo student project: one exercise, one failure, free tools, and a normal laptop.

References

[1] National Science Foundation. Training-based Workforce Development for Advanced Cyberinfrastructure (CyberTraining). https://www.nsf.gov/funding/opportunities/training-based-workforce-development-advanced

[2] Irvine, C. E., Thompson, M. F., McCarrin, M., and Khosalim, J. Labtainers: A Docker-based Framework for Cybersecurity Labs. USENIX Workshop on Advances in Security Education, 2017. https://usenix.org/conference/ase17/workshop-program/presentation/irvine